

breaking traditional silos. Figure 5 shows the different high-level phases of implementing CI/CD.

Jenkins vs GitLab CI vs Azure DevOps

To conclude this article, let's

understand the difference between Jenkins, GitLab CI, and Azure DevOps.

	Jenkins	GitLab CI	Azure DevOps
Integration with other tools	Adaptable using plugins	Good for GitLab and other tools integration	Good with Microsoft products and adaptable to other tools
Ease of use	Easy to learn for beginners, but difficult to maintain and manage	Simple to use, but complicated to maintain and manage if you're using the self-hosted Community Edition	Easy to use, since it's in the form of SaaS; there are minimal management and maintenance efforts
Scalability	High controller – agent architecture	High runners	High on-premise agents and cloud agents
Pricing	Free	Community/Commercial	Pay-as-you-go
Quick start for usage	No; it is necessary to install and configure	No; it is necessary to install and configure	Yes; create an account and you are ready

END 

References

- <https://www.redhat.com/en/topics/devops/what-is-ci-cd>
- <https://www.jenkins.io/>
- <https://docs.gitlab.com/ee/ci/>
- <https://azure.microsoft.com/en-in/products/devops/pipelines/>
- https://about.gitlab.com/direction/verify/continuous_integration/azure_devops_detail.html

By: Ruby Verma

The author has more than 13 years of experience in software development and loves to learn and explore new technologies.

Continued from page...48

Adding actions: Leverage pre-built actions from the GitHub Marketplace or create custom actions to perform specific tasks within your workflow.

steps:

```
- name: Checkout Code
  uses: actions/checkout@v2

- name: Set up Node.js
  uses: actions/setup-node@v2
  with:
    node-version: '14'
```

Reviewing and iterating: Test your workflows locally using tools like *act* or simulate events within GitHub to ensure they behave as expected. Continuously review and refine your workflows based on feedback and evolving project requirements.

Monitoring and optimising: Monitor workflow executions through the GitHub Actions interface, review logs, and optimise workflows to improve efficiency and reliability.

Identify bottlenecks and areas for improvement to ensure optimal performance.

GitHub Actions empower development teams to adopt DevOps practices effectively by automating workflows, enhancing collaboration, and accelerating the software delivery pipeline. By leveraging GitHub Actions, organisations can achieve faster time-to-market, improved code quality, and overall agility in software development.

Whether you're a developer aiming to streamline your CI/CD processes or a team lead looking to enhance DevOps practices within your organisation, GitHub Actions provide a robust platform to automate and optimise your workflows seamlessly. Embrace the power of GitHub Actions to accelerate your DevOps journey and drive continuous improvement in your software development lifecycle. 

By: Dr Shubham Mahajan

The author is assistant professor at Christ (Deemed to be University), Delhi NCR. His research interests primarily lie in image processing, video compression, image segmentation, fuzzy entropy, data mining, machine learning, robotics.